Code Explanation

BNCPLS IF23B 10K File Generation Pipeline Code Explanation

The BNCPLS IF23B 10K File Generation pipeline is a Python-based data processing pipeline that generates files based on data extracted from Oracle tables. The pipeline is built using Apache Spark and involves several stages, including data extraction, AURA payload processing, field calculations, and file splitting.

Overview of the Code
The code is organized into several functions that perform specific tasks within the pipeline. It starts by parsing command-line arguments, creating a Spark session, and then executing the pipeline. The pipeline involves extracting data from Oracle tables, processing AURA payloads, applying field calculations, and splitting the data into files.

Step 1: Parsing Command Line Arguments
This step involves parsing the command-line arguments required for the pipeline to execute. The arguments include the SQL query for source data, paths to XSLT files, output directory, file prefix, file extension, batch ID, run ID, maximum rows per file, and logging level.

```
def parse_arguments() -> PipelineConfig:
    parser = argparse.ArgumentParser(description='BNCPLS IF23B 10K F
ile Generator')

    parser.add_argument('--src_sql', required=True, help='SQL query
for source data')
    parser.add_argument('--xslt_file', required=True, help='Path to
XSLT file for Aura 14')
    # ... other arguments

    args = parser.parse_args()

    return PipelineConfig(
        src_sql=args.src_sql,
        xslt_file=args.xslt_file,
        # ... other config properties
    )
```

Step 2: Creating a Spark Session
This step creates a Spark session with specific configurations required for the pipeline.

```
def create_spark_session() -> SparkSession:
    return (SparkSession.builder
            .appName("BNCPLS_IF23B_10K_File_Generator")
            .config("spark.sql.legacy.timeParserPolicy", "LEGACY")
            # ... other Spark configurations
            .getOrCreate())
```

Step 3: Extracting Source Data
This step involves extracting data from Oracle tables using a provided SQL query.

```
def extract_source_data(spark: SparkSession, src_sql: str) -> DataFr
ame:
    source_df = spark.read
        .format("jdbc")
        .option("url", "jdbc:oracle:thin:@//oracle-host:1521/service
_name")
        .option("dbtable", f"({src_sql}) src_data")
        .option("user", "ZSYSBNCPLSDEV")
        .option("password", "********")
        .option("driver", "oracle.jdbc.driver.OracleDriver")
        .load()

    return source_df
```

Step 4: Processing AURA Payloads
This step processes AURA payloads in the extracted data by applying an XSLT transformation.

```
def process_aura_payloads(df: DataFrame, config: PipelineConfig) ->
DataFrame:
    if "I_AURA_INPUT_BASE64" in df.columns:
        result_df = df.withColumn(
            "parsed_aura_result",
            expr("parse_aura_payload(I_AURA_INPUT_BASE64, POLICY_NUM
BER, " +
                 f"'{config.xslt_file}', '{config.aura15_xslt_file}'
)")
        )

        result_df = result_df.withColumn(
            "PARSED_AURA_OUTPUT",
            col("parsed_aura_result").getItem(0)
        ).drop("parsed_aura_result")

        return result_df
    else:
        return df
```

Step 5: Applying Field Calculations
This step applies field calculations to the processed data.

```
def apply_field_calculations(df: DataFrame) -> DataFrame:
    # Implementation of field calculations (mapplet logic)
    # For demonstration purposes, this function is not shown in deta
il
    pass
```

Step 6: Splitting Data into Files
This step splits the processed data into files based on a specified maximum row count.

```
def split_dataframe_by_row_count(
    spark: SparkSession,
```

```
    df: DataFrame,
    output_dir: str,
    file_prefix: str,
    batch_id: str,
    file_ext: str,
    max_rows_per_file: int
) -> List[Dict[str, Any]]:
    # Implementation of splitting data into files
    # For demonstration purposes, this function is not shown in deta
il
    pass
```

Step 7: Running the Pipeline

This step orchestrates the entire pipeline by calling the functions in sequence.

```
def run_pipeline(config: PipelineConfig) -> Dict[str, Any]:
    spark = create_spark_session()

    try:
        source_df = extract_source_data(spark, config.src_sql)
        df_with_aura = process_aura_payloads(source_df, config)
        df_with_fields = apply_field_calculations(df_with_aura)
        file_info = split_dataframe_by_row_count(
            spark,
            df_with_fields,
            config.output_dir,
            config.file_prefix,
            config.batch_id,
            config.file_ext,
            config.max_rows_per_file
        )

        # Summarize results
        total_records = sum(info["row_count"] for info in file_info)
        total_files = len(file_info)

        result = {
            "status": "SUCCESS",
            "total_records": total_records,
            "total_files": total_files,
            "file_info": file_info
        }

        return result

    except Exception as e:
        logger.error(f"Pipeline failed: {str(e)}", exc_info=True)
        return {
            "status": "FAILED",
            "error": str(e)
        }
    finally:
        spark.stop()
```

Step 8: Executing the Pipeline

The final step involves parsing command-line arguments, setting the logging level, running the pipeline, and exiting with an appropriate status code.

```
if __name__ == "__main__":
    config = parse_arguments()
    logging.getLogger().setLevel(getattr(logging, config.log_level))
    result = run_pipeline(config)
    sys.exit(0 if result["status"] == "SUCCESS" else 1)
```